Strata — Developer Guide

For engineers implementing tickets

strata-kb v1.35.0 · 2026-11-08

1. Who this guide is for

You write product code. Tickets reach you already grounded in the organisation's knowledge base, with citations pinned to a specific version of a specific document, and you implement them.

Strata gives you **the exact section the analyst cited**, at the revision they read, with a warning if it has changed since. It also **publishes what your codebase is**, so analysts, other teams and future you can search your services, dependencies, endpoints and schema without reading your repository.

A dev repo does	A dev repo does not
Read the hub's published knowledge, including the BA's published tickets	Ingest documents from outside the repo
Publish knowledge about **its own code**	Publish domain knowledge
Run an implementation workflow of slash commands	Merge or ship anything by itself (unless you turn on auto mode, section 4)

It publishes two documents:

Document	Origin	Needs review
`<repo_id>-code`	`kb code-ingest`, deterministic, no model. Regenerated on demand from the working tree, never committed	no
`<repo_id>-svc`	drafted by a model from that evidence, corrected by you	always

`-code` is machine fact: services, technologies, dependencies, commands, endpoints, schema, folders. `-svc` is human claim: what each service is *responsible for*. They share section ids (`svc.<name>` exists in both), so an analyst drawing a C4 container gets the alias, label and technology from `-code` and the description from `-svc`.

Shared concepts, such as the L0 to L3 layers and `kb-context` citations, are explained in the architecture document. The analyst's side is in the BA guide.

Slash commands. Every slash command in this guide exists for Claude Code, GitHub Copilot and Cursor, with these exceptions:

- `/kb-summarize` has no Copilot prompt. Copilot loads `.github/instructions/kb-summarize.instructions.md` while you work in `.kb/**`. Cursor also has a rule, `.cursor/rules/kb-summarize.mdc`.

- `/kb-publish` is a skill in Claude Code (`.claude/skills/kb-publish/`), not a file in `.claude/commands/`. You still type `/kb-publish`.
- A dev repo has no `/kb-ingest`, `/kb-init` or `/kb-l3-fix`: it never ingests outside documents.

---

2. First-time setup

2.1 Install and scaffold

```
pip install "strata-kb[code]"
kb init --kind dev
```

The `code` extra (`tree-sitter` plus `tree-sitter-language-pack`, about 184 MB) is what lets `kb code-ingest` read routes, pages, jobs and symbols out of source code. Without it those sections are reported as `not scanned`, jobs stay `partial`, the SA has to ask you about them, and `kb doctor` warns. CI (`kb-code.yml`) installs the extra for you.

`kb init --kind dev` writes the config, the MCP wiring, the slash commands and skills for all three assistants, the CI workflows (`kb-pr-lint.yml`, `kb-code.yml`, `kb-doctor.yml`), `docs/impl/`, the PR template and a conventions pack for each detected language. Use `--lang <name>` (repeatable) for a language it cannot detect from a manifest. Re-running it is safe (see the appendix).

2.2 Point at the hub and register the repo

```
# .kb/config.yaml
kind: dev
repo_id: payments
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com
```

- **`hub`** is the read source for `kb query`, `kb get`, `kb resolve` and the PR and lint gates.
- **`intake`** is how CI publishes: `kb publish` pushes a `kb-publish/<ts>` tag and `kb-code.yml` runs `kb code-ingest`, `kb build`, `kb ci-publish`, which authenticates to the intake with GitHub Actions OIDC. No secret lives in this repo.
- **`repo_id`** must equal this repo's id in the hub's `federation/registry.yaml`. Ask the hub maintainer to add the repo under `repos:`; publishing is refused until they do. Branch protection on the hub plus the OIDC intake are the actual security boundary.

Never put a token in `hub:` or `intake:`. The file is committed, so the token lands in history. `kb doctor` warns; rewrite the URL, pass the credentialed URL in the `STRATA_KB_HUB` environment variable (it overrides the file), and rotate the token. Any machine that uses a credentialed hub URL needs Git 2.31 or newer.

`hub:` and `STRATA_KB_HUB_URL` are different values: `hub:` is the git URL or path of the hub repo; `STRATA_KB_HUB_URL` is the hub's HTTP base URL, used by MCP, claims and the auto runner.

If the KB should carry a SQLite schema or extra index tags, say so here, not in `kb-code.yml`, so every local refresh and CI run ingest the same way:

```
code_ingest:
 db: [data/app.sqlite]   # explicit only
```

```
tags: [payments]
```

2.3 Connect your assistant

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) point at the hub through two placeholders, `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}`. Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token, writes both into `.env`, makes sure git ignores that file, and verifies them against the hub, so a wrong URL and a rejected token come back as different errors. Get the token from the hub maintainer.

Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` variable over `--token`. That flag leaves the token in your shell history and in the process listing that other users on the machine can read. `kb mcp-setup` also warns when the URL is plain `http://` to a non-loopback host, because the token then crosses the network in clear.

The MCP config reads those variables from your environment, not from `.env`, at the moment your assistant starts. Load the file (`set -a; source .env; set +a`, or use direnv) and restart the assistant.

One version for every repo. Every repo runs the hub's strata-kb version. Upgrade the hub first. Then, once `kb mcp-setup` has run (above), run `kb doctor` in this repo: it installs the hub's version, sets the `strata-kb==<hub version>` pins in `.github/workflows/`, and refreshes the skills and commands. In CI it never installs, re-pins or refreshes skills; a CI job that runs `kb doctor` with `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` set reports a version mismatch as an error instead (none of the scaffolded workflows runs it). With `STRATA_KB_HUB` or `--hub` set, `kb doctor` skips the MCP path and the version check, and only reports that the version was not checked against the hub. The files under `.github/instructions/` and `.cursor/rules/` (the Copilot and Cursor `kb-summarize` files and the per-language coding rules) refresh only through `kb init`. An unpinned `pip install strata-kb` in a job that holds `id-token: write` would let every run pick up whatever PyPI serves, in a job able to write to the hub; that is why the workflows are pinned.

2.4 Turn on the PR gate and the daily sweep

`kb init` writes `.github/workflows/kb-pr-lint.yml` but cannot enable branch protection. Add **`pr-lint` to the branch's required checks**. Lint running in CI does not block a merge by itself; requiring the check does.

> Decide this knowingly. Unlike the BA repo's gate, this one never self-skips. Once required, it blocks **every** pull request lacking the required sections, including bot pull requests such as a dependency bump or a revert.

`kb-doctor.yml` sweeps your tickets' citations once a day and keeps one `kb-stale-citations` issue. It needs the Actions **variable** `STRATA_KB_HUB` and, for a private hub only, the Actions **secret** `KB_HUB_TOKEN` (a read token).

2.5 Ask whether the hub auto-merges your repo

Auto-merge is a per-repo switch on the hub's Registry page, turned on by a hub admin. Ask your hub maintainer whether it is on, and know what it does:

> `kb ci-publish` opens **one** hub pull request per push, and the intake reuses the publish branch while a PR is pending, so a regenerated `-code` and an already-merged `-svc` amend land **both** in that PR. With the switch on, the hub merges the **whole** PR right after opening it, `-svc` content included. The `-svc` review therefore happens locally, in `/dev-code-seed` step 4, before the section reaches `reviewed`, never at the hub.

If the merge fails, the PR stays open and the publish still counts as done. With the switch off, every push adds to one pending hub PR that someone has to merge, and `kb doctor` reports when the hub's `-code` lags.

2.6 Seed `-svc` for an existing project (once)

Your repository already has services doing real work. Do not document them from a blank page: run the one-time seed.

```
/dev-code-seed
```

#	Step	Who
1	**Preflight**: `hub:`, `repo_id:`, `intake:` set, repo registered on the hub, clean tree	agent
2	**Extract**: `kb code-ingest --scaffold-svc` regenerates `-code` and creates a `pending svc.<name>` per detected service	agent
3	**Draft**: `kb summarize <repo_id>-svc` writes a responsibility paragraph for each	agent
4	**Review**: walk every draft against its code evidence and correct it	**you**
5	**Approve**: `kb approve <repo_id>-svc` flips the corrected sections to `reviewed`	you
6	**Flows** (optional): add `flow.<name>` sections for business flows across services	you
7	**Validate and publish**: `kb build` without `--allow-pending`, then `kb publish --pr`	you

> **The model's draft is a draft, not a fact.** It can name the wrong flow, miss a responsibility or over-claim one. Never approve a section you have not read and corrected: approving it unread defeats the gate. Where a responsibility touches a domain standard, cite `doc-id §section` from the hub instead of restating the rule.

Budget 10 to 15 minutes per service for step 4; it is a one-time cost. Skip step 6 freely: a missing flow beats a guessed one. Use `kb publish --pr` for this first publish: no merge has happened yet, so there is no tag for `kb-code.yml` to react to, and plain `kb publish` would wait up to ten minutes for a CI run that never starts.

To correct one service, edit its section by hand. `kb summarize <repo_id>-svc --redo --include-reviewed` removes the text and sign-off of every reviewed section of the `-svc` document (add `--section <id>` to reset one) and re-summarizes in the same command; the prose you corrected comes back only from git.

---

3. Daily work

3.1 Implement a ticket

All entry points are slash commands. Pick the one that matches where the work stands; every phase resumes cold, because state lives in `docs/impl/<id>/`, the branch and the PR, never in the conversation.

Situation	Run	Result
New ticket, nothing started (small ones collapse themselves)	`/dev-implement-ticket <ticket>`	claims the ticket, fetches it, checks it, then walks design, plan, execute, handover
Design approved, no plan yet	`/dev-plan <id>`	`docs/impl/<id>/plan.md`
Plan approved, or execution in progress	`/dev-execute <id>`	code, task by task
Code written by hand, needs a PR write-up	`/dev-handover <id>`	the PR body, the `kb svc note` history rows
Lost track of where a ticket stands	`/dev-implement-ticket <id>`	detects how far it got and offers the next step
A technical question needing evidence	`/dev-sa <question>`	a decision with `path:line` / `doc §section` evidence
Just checking a citation	`kb resolve <file>`	`ok`, `stale` or `broken` per reference

A ticket id is enough. Intake gets the ticket **from the hub**: the BA publishes every ready ticket in a `<repo_id>-plan` document, so the hub is the source of truth.

```
kb ticket fetch M-checkout-US3          # or <repo>:<id> if several repos publish it
kb claim take M-checkout-US3            # one Dev per ticket; exit 3 = taken
```

`kb ticket fetch` writes `docs/impl/<id>/ticket.md` (`--out` to change it) and overwrites it on every fetch. The copy is derivable from the hub, so `docs/impl/.gitignore` ignores it: never commit it and never edit it. If the ticket is wrong, the BA amends it in the BA repo. It exits `1` when no published plan has the id (ask the BA to publish) or when several do (qualify it as `<repo>:<id>`). The hub copy has no `## Definition of Ready`: being on the hub means the BA ticked every box.

Right after Intake the flow runs `kb ticket check docs/impl/<id>/ticket.md` (skipped when the ticket has no `## Technical grounding`):

Output	Meaning	What to do
PASS, `[note] ... still valid`	Nothing the ticket cites changed	Carry on
PASS, `[warn] ... review the diff`	A cited section changed, every cited name still holds	Read the diff; carry on unless it changes the ticket's meaning
FAIL, `re-run /sa-ticket-ground`	A cited id, column, route, command or file is gone	Stop. It is a blocker for the BA, whose SA re-grounds the ticket
FAIL, `checked against the local -code`	The hub was unreachable or lacks `-code`; your branch may be the cause	Re-run when the hub is reachable
FAIL, no re-ground line	An open decision or an undecided `[NEW: D<k>]`	Report it to the BA and stop

The hub's Ticket menu (`/ui/tickets`) and `kb mission next --from-hub [--repo <ba-repo>]` show each story as **done**, **ready**, **not ready** or **blocked**. Pick a **ready** one. A `draft` ticket whose reason says the parent mission was amended is not published: wait for the BA, and do not implement from an older copy.

3.2 Four human gates, five agent reviews

```
/dev-implement-ticket <ticket>     intake, questions, check, resolve, ground
       |
       +-> dev-design    -- A1 review -- GATE 1: you approve the design
       +-> dev-plan      -- A2 review -- GATE 2: you approve the plan
       +-> dev-execute      A3 per task, A4 at the end (repeatable, resumable)
       +-> dev-handover  -- A5 merge-risk review -- GATE 3: you open the PR
                                                    GATE 4: you merge it
```

Nothing merges or ships without a human. The agent never opens the PR or merges. Before each of the first three gates a different agent has read the work back in a context of its own, never the agent that produced it:

	Where	What it looks at
A1	`dev-design`, before GATE 1	does the design match the ticket
A2	`dev-plan`, before GATE 2	is the plan test-first and runnable
A3	`dev-execute`, per task	does the diff meet its spec, and is the code good
A4	`dev-execute`, after the last task	does the finished branch round out the ticket
A5	`dev-handover`, before GATE 3	is merging into the default branch safe

Criteria live in `docs/pr-review-rubric.md`, overridden by `docs/pr-review-rubric.local.md`. The round limits are in the skills; a finding still open after the last round stops the flow and hands it to you. Where your assistant cannot dispatch subagents (the default for the Copilot and Cursor wrappers) the review runs as a single pass and is a **report, not a gate**: the phase stops and hands you the result. `dev-execute` orders tasks with `kb plan waves docs/impl/<id>/plan.md` and runs independent ones in parallel git worktree lanes. `kb plan waves` fails on a shared file without a dependency, a cycle, a task that lists no paths under **Files**, a missing `Depends on:` line or one that names a task that does not exist, or an empty `Serial resource:` line; a task with a `Serial resource:` line prints after `serial:` and runs alone, never in a parallel lane.

3.3 Questions for the BA

`dev-design` and `dev-plan` never settle a doubt by guessing. Each doubt is tagged `business` or `technical`:

- **Technical** (how to build it, where it goes) goes to `dev-sa`, the solution architect of this repo. It reads the source and the hub's `-code`, `-svc` and domain documents, and the decision lands in the draft as `- SA-<n>: <decision> — why — evidence`. You can call it yourself: `/dev-sa <question>`.
- **Business** goes to the BA through hub Q&A (MCP `kb_question_ask`) and appears as `- Q<id> — <question> — blocks: <AC ids|none>`. The BA answers with `/ba-answer-questions`.

A question that blocks an acceptance criterion stops the gate: the skill gives you the hub's Q&A link to paste in chat. On every entry `/dev-implement-ticket` reads the ticket's questions and shows `BA-Q: none | open Q<ids> | waiting Q<ids>`; a row that is *open* or *waiting* and reads `blocks this ticket: yes` stops the flow; any other row is information only. Q&A works only when MCP points at the hub's HTTP server (`kb mcp-setup`).

Questions on a split story. A question about your child ticket is recorded on the parent story, and the list for your ticket includes the parent's questions. Each row reads `blocks this ticket: yes` or `no`. The gates stop only on an open or waiting row that reads `yes` (`kb_question_list(ticket=<id>, status="all")`). A question that the BA only *proposed* to block your ticket does not stop you until the BA confirms it. A parent question reads *answered* only once the parent and every affected child are republished on the hub.

Pre-merge re-check. Right before you open the PR or merge (and in Auto mode before `kb pr merge`), `dev-handover` lists the questions again. A blocking open or waiting row stops the merge (Auto: `blocked-qa Q<id>`). It then fetches the ticket to a scratch path and compares its `> Revision:` with `docs/impl/<id>/ticket.md`. A higher revision stops the merge (Auto: `failed ticket-revised <ticket-id>`): the BA changed the ticket while you built.

3.4 Fix a bug

A bug the BA published is `BUG-<n>`: no story, no acceptance criteria, done when its steps no longer reproduce the wrong result. `/dev-fix-bug BUG-<n>` runs:

1. **Intake**: `kb ticket fetch BUG-<n>` (also copies the screenshots to `docs/impl/<id>/assets/`); branch `fix/<BUG-id>`, or `hotfix/<BUG-id>`.
2. **Reproduce**: a failing test that shows the Actual result. No fix without it.
3. **Root cause**: hypotheses in `docs/impl/<id>/rootcause.md`; three failed ones stop the flow for an architecture talk with you.
4. **Fix**: one change at the root cause, full suite green.
5. **Handover**: `/dev-handover` with bug rules; the reproduction test and root cause replace the design and plan.

Bugs across repos

You are the first responder when the bug's `> Target repo:` is your repo. If the root cause also sits in another repo, propose a split.

1. Write a `## Child bug proposal` in `docs/impl/<id>/rootcause.md`: one child per repo, the blockers, and whether your repo has an `Own fix`.
2. Ask `Split proposal for BUG-<n>: <one line>` on the hub.
3. The run stops your claim (`--stopped "split into child bugs"`) and ends `AUTO-RESULT: blocked-split BUG-<n>`.

Run `/dev-fix-bug BUG-<n>` again later. It reads the hub first.

- Parent bug now: you do the own fix, once its blockers are done.
- Question answered, no split: keep fixing.
- Question still open or waiting: the claim is stopped again and the run waits.
- No question: it asks.

Each child bug goes to the Dev of its repo. The parent bug is done when every child is done and, with `Own fix: yes`, your own fix is done. A `-svc` entry for `BUG-<n>` means the own fix only.

3.5 Several Devs on one repo: claims

A **ticket claim** on the hub keeps two Devs off the same ticket. `/dev-implement-ticket` and `/dev-fix-bug` run `kb claim take <id>`, and every later phase refreshes it.

- The claim names its holder (`<git user.name>@<hostname>`, or `STRATA_KB_CLAIM_AS` or `--as`) and a phase (`claimed`, `design`, `plan`, `execute`, `handover`, `waiting-qa`, `review`, `publishing`). A second Dev gets `taken by <holder> (<phase>, updated <x> ago)` and exit `3`: pick another ticket or ask a hub admin to release it.
- By default it expires after 4 hours without an update. Once you have opened the PR, run `/dev-handover <id>` again: it sets `review` with the PR URL. A claim in `review` or `publishing` with a PR, or marked `stopped` by the auto runner, never expires. If your claim is released while you work, the next update exits `3` (`lost: <reason> by <who>`) and the skill stops without pushing.
- `kb claim list [--json]`, `kb claim release <ticket> [--reason TEXT]` and `kb claim update <ticket> --phase <p>` manage claims. These commands need `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN`.

3.6 Publish after the merge

A push to the default branch publishes nothing; the intake accepts a publish only from a `kb-publish/*` tag. After your PR merges, publish from your default branch (`<default>`):

```
git fetch origin && git switch <default> && git pull --ff-only origin <default>
kb publish
```

In a dev repo `kb publish` refuses to run from a checkout that is behind `origin/<default>`, because the snapshot would drop another machine's newer `hist` row. The messages are `behind origin/<default> — pull, then kb publish`, `no origin remote — ...` and `could not fetch origin/<default>: ...`. A squash merge leaves the ticket branch behind, which is why you switch first. The hub reflects the commit once its PR merges. `kb publish` and `kb ci-publish` warn how many sections ship unreviewed (`--require-reviewed` makes that fatal).

3.7 Work with citations

```
kb resolve tickets/ABC-123.md    # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>
kb get hr-handbook 4.12 --level l3
```

Status	Meaning	What to do
`ok`	The pinned content still matches what is published	Implement against it
`stale`	The source was amended after the ticket was written	Run `kb diff`, then ask the analyst whether the acceptance criteria still hold. **Do not silently implement the new version.**
`broken`	The reference no longer resolves	Escalate. The ticket is not implementable as written

Every entry point re-checks freshness first, because the hub may have published since your last session. When a value in your code comes from a standard (a format, an enum, a threshold, a length), take it from the resolved section at the pinned version and leave a citation comment next to it.

3.8 Keep knowledge current

Neither document needs new per-ticket discipline from you.

- **`-code` needs nothing.** It is never committed (`.gitignore` carries `.kb/*-code/`). `kb svc note`, `kb build`, `kb doctor` and `kb publish` regenerate it first when it is missing, older than `HEAD`, or your tracked files changed (`--no-refresh` skips that). CI never passes `--scaffold-svc`, because it must never create `pending` content.
- **`-svc` accrues at handover.** `dev-handover` runs `kb svc note payments-api --ticket ABC-123 --title "Add refund endpoint"` for every service a ticket touched. It appends one `hist.<service>` row, idempotently; never edit `hist.*`.
- **Drift is reported, not repaired.** When refreshed evidence of a `reviewed` service changes, `kb code-ingest --scaffold-svc` prints `stale-risk: svc.<name>` and leaves the summary alone.

---

4. Automation modes

Full-auto mode (optional). A switch per `ba` and `dev` repo that lets the two repos run on their own, on your machine, until every ticket on the hub is done. No step waits for a person: the agent reviews its own work, approves GATE 1 and GATE 2, picks the best option, writes down why in `docs/impl/<id>/auto-decisions.md` (copied into the PR's `## Notes`), opens the PR, fixes CI until it is green, merges and runs `kb publish`. It is built for **one BA repo and one Dev repo per hub**. Nothing changes while the switch is off.

4.1 Security: read this first

The permission deny list that `kb init` writes (appendix) is defence in depth, not a boundary. An unattended run merges its own pull requests and publishes to the hub with no human review. A ticket the agent misreads goes straight to `main` and to the hub. `kb auto on` prints this itself: "Auto mode merges pull requests as admin, with no review. Turn it on only for a repo where that is acceptable."

It merges with `kb pr merge` (GitHub REST), using the rights of your `gh` login; `--admin` is accepted for `gh` compatibility and changes nothing. If that login is a repo admin and branch protection does not include administrators, those merges bypass the protection. So, before you run it, the repo owner **must** enable GitHub branch protection on `main`:

- block force pushes and deletions;
- **include administrators**;
- keep the CI check as a **required status check**: the `test-<lang>` job(s) of `kb-test.yml` (or of the workflow carrying the `kb-test` marker). Branch protection picks checks by job name. Without a required check, "green CI" is only a convention the agent follows;
- do **not** require approving reviews: the agent cannot approve its own PR, and the merge is then refused. If you require reviews anyway, exclude administrators from that rule and accept that the agent bypasses it on `main`.

The deny list also cannot see the current branch: the agent's working tree must never sit on `main` with commits on it. The skills branch first; the runner does not guard it. The runner never uses `--dangerously-skip-permissions` or `bypassPermissions`.

4.2 Set up once

1. A mission and its tickets are on the hub (the BA side, by hand or in auto mode).

2. `intake:` is set in `.kb/config.yaml`, and the hub's Registry page has **auto-merge on** for this repo, so the `hist` row lands and stories reach `done`. This applies to the BA repo too.
3. Re-run `kb init --kind dev`. It merges the permission allowlist, including the test, install, lint and build commands for the `langs:` recorded in `.kb/config.yaml`, into `.claude/settings.json`, and refreshes the skills and `/dev-test-workflow`.
4. Export `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN`, and have the `claude` CLI on the `PATH`, **logged in with a Claude subscription**. The runner never uses an API key.
5. Keep `main` as the default branch; the skills run `git fetch origin main`.

4.3 Run and stop

Task	Command	Result
Turn the switch on	`kb auto on`	prints the warnings, flips `auto.enabled`
Commit it	commit and push `.kb/config.yaml` and `.claude/settings.json` to the default branch	`kb auto run` refuses until you do
Start the loop	`kb auto run` (`--once`, `--model`, `--effort`), or `/kb-auto run`	`/kb-auto run` does all three steps and starts the loop detached; log in `.kb/auto/run.log`
See where it is	`kb auto status [--json]`, `/kb-auto status`	switch, task, attempts, questions waited on, stop reason
Stop it	`kb auto off`, `/kb-auto off`	stops after the current task
Wait for a result	`kb auto wait [--max 6600] [--interval 30]`	one line, used by the cloud watcher

`kb auto run` refuses, and prints every reason, when the switch is off, `.kb/config.yaml` or `.claude/settings.json` is untracked or dirty, `auto.enabled: true` is missing from the default branch's copy of the config, `intake:` is empty, or the hub registry cannot be read, does not list this repo, or lacks `auto_merge: true` for it. Exit codes: `0` done, `2` stopped, `1` refused or error, `130` interrupted.

4.4 What it does and when it stops

One fresh `claude -p` process per task. In order:

1. If no workflow runs the repo's tests, `/dev-test-workflow` writes and merges `kb-test.yml`, so "green CI" means tests ran.
2. Bugs first (`/dev-fix-bug`).
3. The first `ready` ticket that has no mission and is not a bug (`/dev-implement-ticket`).
4. The first `ready` story in mission order whose dependencies are `done`.

The runner takes only tickets the hub routes to this repo's `repo_id`: the ticket's `> Target repo:` header, else the repo of its first qualified `Grounded on:`; with one dev repo on the hub, every ticket without a target. A ticket aimed at another repo is never picked here, and the "all done" exit counts only this repo's tickets and the untargeted ones. The hub plan shows why a ticket waits: `no target repo`, `target <x> is not a dev repo on the hub`, `blocked by <id> (not done)` or `dependency cycle: ...`. Done is read from the target repo's `<repo>-svc` history; while that repo has not published, the plan says `done: unknown (<repo>-svc not published)`. If the task the runner stored vanished from the plan or moved to another repo, the runner drops it, releases its hub claim and picks again. `kb ticket fetch` warns `<id> targets <x>, this repo is <repo_id>` when you fetch a ticket of another repo.

Upgrade order. Upgrade the hub first, then every Dev runner. A runner refuses to start against a hub older than 1.35 (`hub is older than 1.35 — upgrade the hub first`), and `kb publish` warns `hub is older than 1.35: Dev runners will not route by repo`. Add `> Target repo:` or qualified grounding to open tickets before registering a second dev repo: from then on untargeted tickets are blocked.

When a business question blocks a ticket, the Dev asks the BA through hub Q&A and waits; once answered, the same ticket resumes in a fresh process. It stops after two failed attempts at one ticket (a red CI after two fix rounds stops it at once); fix the cause and run `kb auto run` again. A command outside the allowlist ends the task `failed command-denied <command>`: add it to `permissions.allow` in `.claude/settings.json` by hand. A usage-limit wait (15, 30, 60 minutes) is not a failure. It finishes, exit `0`, when every story, unparented ticket and bug is `done`. A task process carries `STRATA_KB_AUTO_TASK=1`, under which `kb auto on`, `off` and `run` refuse (`status` still works), so a task can neither flip the switch nor start a second loop.

The runner's prompt asks each task to end its reply with one line: `AUTO-RESULT: done`, `AUTO-RESULT: blocked-qa Q<id>[,Q<id>...]` or `AUTO-RESULT: failed <reason>`, and nothing after it. A missing or malformed line counts as a failed attempt (`no result line`, `bad result line`). A skill you write for the runner must end the same way.

4.5 Claude Code cloud

`/kb-auto run` also works in a Claude Code cloud session (detected from `CLAUDE_CODE_REMOTE`; `STRATA_KB_CLOUD=1|0` forces it). Set `STRATA_KB_HUB_URL` and `STRATA_KB_HTTP_TOKEN` in the cloud environment. The hub is read live through its API and nothing is cloned; other hub-reading `kb` commands refuse and point at the `strata-kb` MCP tools. `kb pr` talks to the GitHub REST API instead of `gh pr`, because GraphQL is blocked there. Because nobody can click "trust this folder", `kb auto run` there marks the repo root trusted in `~/.claude.json`; its other keys are kept and an unparseable file is left alone.

The session branch cannot push to the default branch. When the default branch's switch is off, `/kb-auto run` opens an `auto/switch` PR that flips only `auto.enabled`, waits for green checks, and then **merges that PR itself, telling you first**, with `kb pr merge auto/switch --squash --admin --delete-branch`. The same branch-protection advice as in 4.1 applies: that merge has no review either. `/kb-auto run` is re-runnable: it removes a leftover local `auto/switch` worktree and branch itself. A switch PR left open by a failed run (red checks, merge error) and its remote `auto/switch` branch must be closed and deleted on GitHub by hand first; the skill stops and says so, and never force-pushes. **The session must stay open** for the whole run. Not yet verified on real cloud: a session lasting over three hours, pushing `auto/*` branches, a merge past a required review, and `gh run` over REST.

---

5. Troubleshooting



6. Appendix

6.1 Config keys and environment variables

Key or variable	Meaning
`kind`, `hub`, `repo_id`, `intake`	see 2.2. Never put a token in `hub:` or `intake:`
`langs`	languages for the conventions pack and the auto-mode allowlist; set by `--lang` or detection
`code_ingest.db`, `code_ingest.tags`	SQLite files (explicit only) and extra index tags for every `-code` refresh
`auto.enabled`, `auto.qa_interval`, `auto.idle_interval`, `auto.hotfix_base`, `auto.claim_heartbeat`	auto mode; `hotfix_base` empty = default branch (the skills currently assume `main`); `claim_heartbeat` is seconds between claim refreshes, default `900`
`kb init` re-run	creates missing files and refreshes scaffold files; `.kb/config.yaml`, `.kb/index.yaml` and `.claude/settings.json` are protected (`--force` replaces them; `.claude/settings.json` is merged otherwise); `*.local.md` files are never touched
`.kb/index.yaml` `llm:`	`runner` (`auto`, `claude`, `copilot`, `none`), `model`, `effort`, `max_workers`, `timeout`, used by `kb summarize <repo_id>-svc`
`STRATA_KB_HUB`	overrides `hub:`; carries a credentialed URL in CI (an Actions variable)
`STRATA_KB_HUB_URL`, `STRATA_KB_HTTP_TOKEN`	the hub's HTTP base URL and bearer token: MCP, claims, the auto runner
`KB_HUB_TOKEN`	Actions secret, a read token, private hub only (`kb-doctor.yml`)
`STRATA_KB_CLAIM_AS`	claim holder name
`STRATA_KB_AUTO_TASK`	set by the runner on task processes; `kb auto on`, `off` and `run` refuse under it
`STRATA_KB_CLOUD`, `CLAUDE_CODE_REMOTE`	force or detect Claude Code cloud

Exit codes: `0` ok, `1` error, `2` stale citation (or a usage error), `3` claim taken or lost, `8` `kb pr checks` still pending.

6.2 Work folders and what is git-ignored

Path	Holds	Committed
`docs/impl/<id>/design.md`, `plan.md`	the design and the plan, each with a `## Review record` table	yes
`docs/impl/<id>/ticket.md`	the ticket fetched from the hub	no, never edit
`docs/impl/<id>/context.md`, `review/`, `assets/`	context cache, review artefacts, bug screenshots (may show personal data)	no
`.kb/*-code/`, `.kb/auto/`, `.env`	derived `-code`, the auto runner's state, the MCP credentials	no

`kb init` and `kb impl tidy` move an old flat layout (`docs/impl/<id>-plan.md`, ...) into `docs/impl/<id>/` without overwriting; commit the renames. Never commit `.kb-qa/`: it is the hub's Q&A store, and it is git-ignored on the hub.

6.3 What `kb code-ingest` extracts

```
kb code-ingest                          # defaults are usually right
kb code-ingest --db data/app.sqlite     # name a database explicitly
kb code-ingest --json                   # machine-readable report
```

Eleven extractors, organised by artefact kind rather than language. The first seven read build, deploy and contract files; the last four read source with tree-sitter.

Extractor	Section prefix	Reads
`services`	`svc.<name>`	compose files, Dockerfiles, k8s manifests, solution and workspace files
`deps`	`dep.<ecosystem>`	manifests for Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift, Dart
`commands`	`cmd.<purpose>`	package scripts, Makefiles, tox, shell scripts, CI run steps
`tree`	`struct.tree`	the files Git tracks, so an ignored virtualenv never appears
`schema`	`db.<table>`	SQL migrations, Prisma, Alembic, EF, and explicitly named SQLite files
`integrations`	`int.<name>`	environment-variable **keys** only
`api`	`api.<tag>`	in-repo OpenAPI and Swagger contracts
`routes`	`route.<slug>`	HTTP routes declared in source (needs `[code]`)
`pages`	`page.<slug>`	UI page routes (file-system routers work without `[code]`)
`jobs`	`job.<slug>`	scheduled jobs of framework schedulers (needs `[code]`)
`symbols`	`sym.<slug>`	exported names, enum and object members, CSS custom properties (needs `[code]`)

For each compose service, the `svc.<name>` section of `-code` also lists its named volumes (bind mounts are not named volumes and are left out), its healthcheck command as one line (`disabled` when the service turns it off, `none` when it has none) and its device reservations (`<driver>:<caps>`, from `deploy.resources.reservations.devices`).

`tree` also writes `struct.files.<dir>` (every tracked file under one top-level directory), and every extractor reports into `struct.coverage`. `kb code-ingest` exits `1` when none of `services`, `deps`, `commands`, `schema`, `integrations` and `api` found anything: a folder listing alone is a misconfiguration, not knowledge.

Route frameworks: NestJS, Express, Next.js, Nuxt, Spring Boot, Quarkus, Micronaut, ASP.NET Core, FastAPI, Flask, Django, Gin, Laravel, Symfony, Actix Web, Axum, Rocket, Vapor.

The `tree-sitter-language-pack` pin is exact, so the same commit gives the same bytes on every machine. Only names are emitted; env values, constant values and enum member values never are.

Two security properties. (1) The integrations extractor emits **keys, never values**: the only env file it opens is `.env.example`, `.sample` or `.template`, never a real `.env`, and its pattern has no capture group around the value, so there is nothing to leak. Compose `environment:` blocks are read, but only the key survives. (2) SQLite schema input is **`--db`-only, never inferred**: a stray `.db` or `.sqlite` in the repo is never read unless you name it, so it cannot become published company knowledge by accident.

`struct.coverage` tells the SA how far to trust an absence: one table, `Area | Scope | Framework/language | Status | Detail`, for `routes`, `pages`, `jobs`, `symbols`, `files` and `schema`.

Status	Meaning
`complete`	Every recognised form in that scope was resolved
`partial`	Something was seen but not resolved (a dynamic route path, a file that does not parse); each site is named as `file:line`
`not scanned`	Nothing was looked at; the detail says why (extra missing, no adapter)

`jobs` is never `complete`: a job can live where no adapter looks (`pg_cron`, a hand-written `setInterval`, a crontab), so its row is always `partial` with the detail `jobs outside framework schedulers are not detectable`. For a route, page, symbol, file or a table's partition, index or primary key, the SA writes a new thing as new only when that scope is `complete`: `<thing> in <path> [NEW: D<n>]` for a route, page or symbol, and `<path> [NEW: D<n>]` as its own `Files:` entry for a file. Otherwise it is an `Open decisions` item that quotes the status, and `kb ticket check` enforces it.

Limits: `schema` does not implement a SQL dialect; a statement it cannot apply makes the scope `partial` and names `file:line`. If `.kb/<doc_id>/` holds a document that `kb code-ingest` did not write, it exits 1 and writes nothing (move it or pick another `--doc-id`). A dirty tree prints a `dirty_tree` warning.

6.4 What is enforced, and what is only asked

Rule	Enforced by
The PR carries its evidence	`kb pr lint` in `kb-pr-lint.yml` on every PR: `## Ticket`, `## Changes`, `## Testing` filled in, the pasted test command, a `Blocking: No` line
Unreviewed knowledge cannot reach the hub	`kb build` exits 1 while any `-svc` section is `pending`
The context cache is CLI-owned	`kb resolve --status-only --cache` refuses a cache whose version, refs or block differ

`kb pr lint` takes a file or stdin and never a string option: a PR description is attacker-controlled text, and keeping it out of argv stops a caller from interpolating it into a shell command.

Everything else (test-first, shown verification, GATE 1 and GATE 2, A1 to A4) is prompt discipline that nothing in `kb` measures; the check is you, at the gate. Only A5 has teeth, through the `Blocking:` line. Test-first has one bounded exception: change classes with no observable behaviour, named in `docs/tdd-exemptions.md` and declared in the plan when it is written.

6.5 Auto-mode permissions

`kb init --kind dev` merges its permissions into your `.claude/settings.json`. **Allowed:** everyday `git` (add, commit, status, diff, log, fetch, pull, switch to `auto/*`, `fix/*`, `hotfix/*`, branch, worktree, merge, `push -u origin HEAD`), `gh pr` and `gh run`, `Read`/`Edit`/`Write`, the `strata-kb` MCP tools, many `kb` subcommands, and the test, install, lint and build commands of your `langs`. **Denied:** any `git push` that names `main` or `master`, force-push, `+` and `:` refspecs and `--delete`; `git checkout`, `switch` and `branch` forms that move onto, create, rename or delete `main` or `master`; `git reset --hard`; `gh repo delete`, `gh run delete`; `kb init`; `rm -rf /*`; editing `.claude/settings.json`, `.claude/settings.local.json` and `.git/**`. The `Stop` and `SubagentStop` hooks run `kb usage ingest-transcript --hook-stdin` for cost reports.

Cost reports. The hook adds about 0.6 s per turn (the cost of starting `kb`). `kb usage report` writes `.kb/usage/report.html`, per ticket, phase and model; with `--ticket <id> --md` it prints a Markdown table. A failing hook is silent by design (a `Stop` hook cannot block a turn from ending), so if `.kb/usage/*.jsonl` is not growing, read `.kb/usage/ingest-errors.log`: it is the only place a failure inside `kb` shows. Re-ingesting an ended transcript is safe, because a call already in the ledger is skipped: `kb usage ingest-transcript <path>` backfills one. Run it only on a session that has ended: a call still streaming would be stored with a partial output count, and a stored row never changes. `report.html` is regenerated from the ledger on every run, so the ledger is the record, not the report. **Model tiering** is a recommendation that nothing in `kb` enforces. Model prices come from `usage-prices.yaml` (shipped, overridable per model at `.kb/usage-prices.yaml`), which carries an `effective_date`. A model with no row is reported `unpriced`, never guessed, and fast-mode calls are priced at the standard rate and flagged, so a total that contains them is a lower bound. Switch model at phase boundaries, never inside a phase: a switch discards the prompt cache and pays a fresh cache write, so a few cheap-model turns inside a strong-model session can cost **more** than not tiering. Check the result with `kb usage report`, which breaks spending down by model.

6.6 Hub Q&A rules that touch you

Questions and answers live in the hub's Q&A store, written only through `kb_question_ask` (Dev) and `kb_question_answer` (BA). `asked_by` and `answered_by` are names the caller types (the skills pass `git config user.name`): tracking, not authentication. Limits: 4000 characters for a question or answer, 1000 for context, 100 for names, at most 50 open questions per ticket. The MCP tools your skills call: `kb_search`, `kb_get_section`, `kb_context_new`, `kb_resolve`, `kb_ticket_lint`, `kb_ticket_check`, `kb_ticket_fetch`, `kb_question_ask`, `kb_question_list`.

6.7 Reserved names

`<repo_id>-code` and `<repo_id>-svc` are reserved for this repository's own code knowledge; a domain document must never take either suffix. Section prefixes are a published contract:

Prefix	Document	Owner
`struct.tree`, `struct.files.<dir>`, `struct.coverage`, `db.<table>`, `dep.<eco>`, `int.<name>`, `api.<tag>`, `route.<slug>`, `page.<slug>`, `job.<slug>`, `sym.<slug>`, `cmd.<purpose>`	`-code`	extractor
`svc.<name>`	`-code` and `-svc`	extractor (evidence) and you (model-drafted responsibility)
`flow.<name>`	`-svc`	you
`hist.<name>`	`-svc`	`kb svc note` only